Team 1

КУРСОВИЙ ПРОЄКТ · ASP.NET CORE

New Balance Shop

Інтернет-магазин спортивного одягу та взуття: каталог, кошик, замовлення, кабінет покупця й адмін-функції.

TEAM LEAD
Панчук Вадим Владиславович
alfa40957@gmail.com

FULL-STACK РОЗРОБНИК
Ноцколіні Рафаїл Львович
Backend · Frontend · DevOps · QA
vicitmceo@gmail.com

Репозиторій

Живий сайт

Захист курсового проєкту · 2026
github.com/vicitmceo/NewBalanceShop

Навіщо цей проєкт

Проблема — Рішення — Цільова аудиторія

ПРОБЛЕМА

Складно швидко знайти й купити потрібну пару

- Каталог на великих сайтах перевантажений: десятки розділів, акцій, банерів.
- Щоб оформити замовлення, часто змушують створювати акаунт.
- Розмір, колір і наявність видно не одразу.

Приклад 1. Людина хоче кросівки 42 розміру в сірому кольорі й губить 10 хвилин на фільтри та реєстрацію.
Приклад 2. Покупець кладе товар у кошик, закриває вкладку, і кошик зникає.

→

РІШЕННЯ

Простий магазин: каталог → кошик → замовлення

- Каталог з картками: назва, розмір, колір, ціна, категорія.
- Кошик працює в сесії, замовлення можна оформити без реєстрації.
- Кабінет покупця: профіль та історія замовлень.
- Адмін-функції: керування товарами та користувачами.
- Три мови інтерфейсу: UA / EN / FR.

→

ЦІЛЬОВА АУДИТОРІЯ

Хто користується

- **Покупці 16–40 років**, які носять спортивне взуття й одяг щодня.
- **Любителі бігу й тренувань**, яким важливі розмір і наявність.
- **Адміністратор магазину**, який керує каталогом і користувачами.

Докладніше про персони: слайд «Персони».

SRS: вимоги до системи

13 функціональних вимог + нефункціональні · файл SRS.md у репозиторії

New Balance Shop

Функціональні вимоги (FR) і стан реалізації

ID	ВИМОГА	СТАН
FR-01, 02	Перегляд списку товарів і одного товару	Готово
FR-03	Фільтр/пошук (за категорією; бренд, ціна — ні)	Частково
FR-04, 05, 06	Створення, оновлення, видалення товару	Готово
FR-07, 08	Кошик і оформлення замовлення без реєстрації	Готово
FR-09	Статус замовлення (New … Cancelled)	Готово
FR-10, 11	Кабінет покупця: профіль, історія замовлень	Готово
FR-12, 13	Адмін: список, блокування, видалення користувачів	Готово

12 / 13

FR ПОВНІСТЮ

1

FR ЧАСТКОВО (FR-03)

Нефункціональні вимоги (NFR)

- **Продуктивність:** читання через AsNoTracking, DTO замість сутностей.
- **Масштабованість:** Clean Architecture, окремі проєкти, stateless-сервіси з DI.
- **Безпека:** хеш паролів PBKDF2, перевірка ролі адміна на ендпоінтах користувачів, HttpOnly-сесія.
- **Надійність:** міграції EF Core, 145 автотестів, покриття 86,7%.
- **Зручність:** інтерфейс UA/EN/FR, кошик без обов'язкової реєстрації.

UI Kit

Дизайн-токени та компоненти фронтенду (React, App.css)

New Balance Shop

Кольори

Red
#cf0a2c

Black
#111111

Gray 600
#5c5c5c

Gray 400
#8c8c8c

Gray 200
#e2e2e2

Gray 100
#f4f4f4

Типографіка

Заголовок 800

Заголовок розділу 20/800

Основний текст 13–15 px. Шрифт: Helvetica Neue, Helvetica, Arial, system-ui.

Кнопки

В кошик | Hover / Купити | Вийти

Навігація й мова

Каталог | Мої замовлення | UA

Форми та статуси

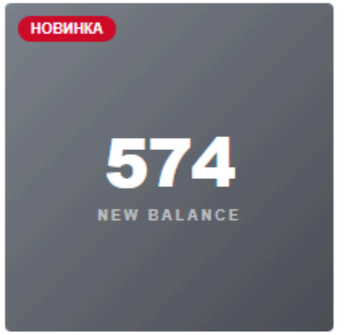
ID покупця

1

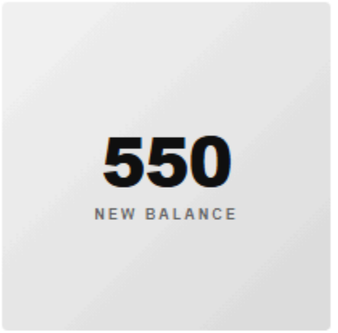
Замовлення #12 оформлено!

Помилка запиту: 401

Картка товару



New Balance 574
Розмір 42 · Сірий
3 299 ₴



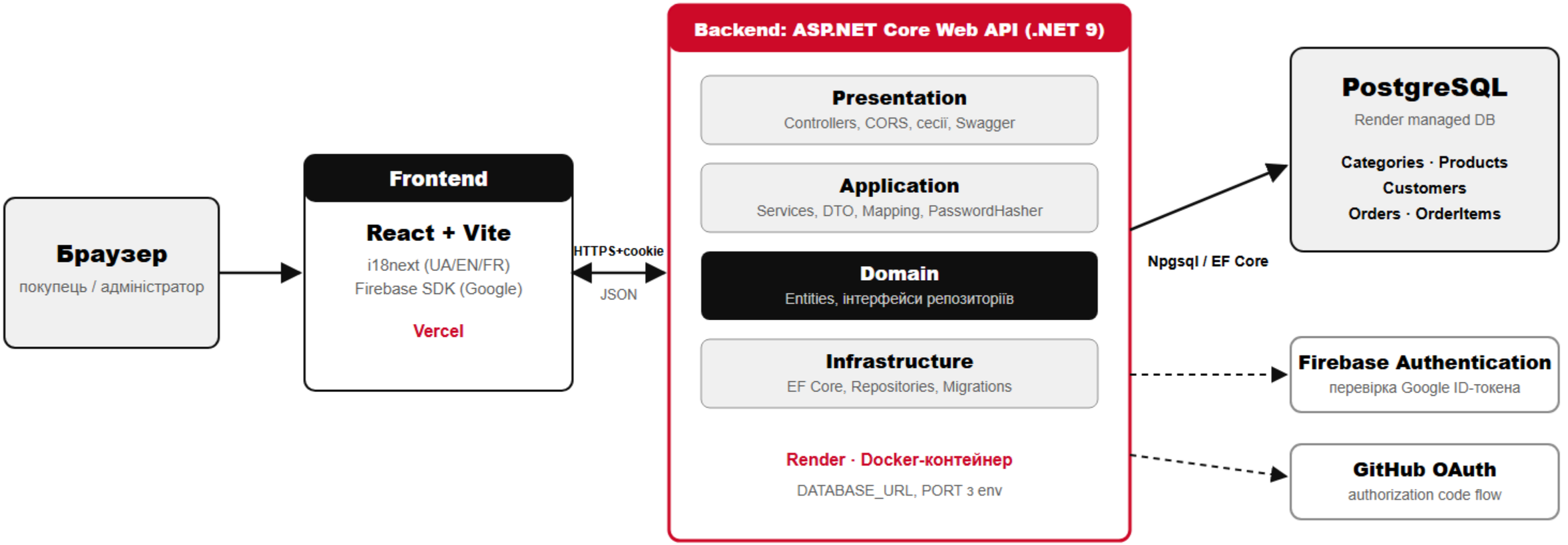
New Balance 550
Розмір 43 · Білий
4 199 ₴

Фото товарів замінено типографськими плитками: немає прав на офіційні фото бренду.

Архітектура системи і розміщення

Фронтенд, бекенд, база даних та зовнішні сервіси

New Balance Shop

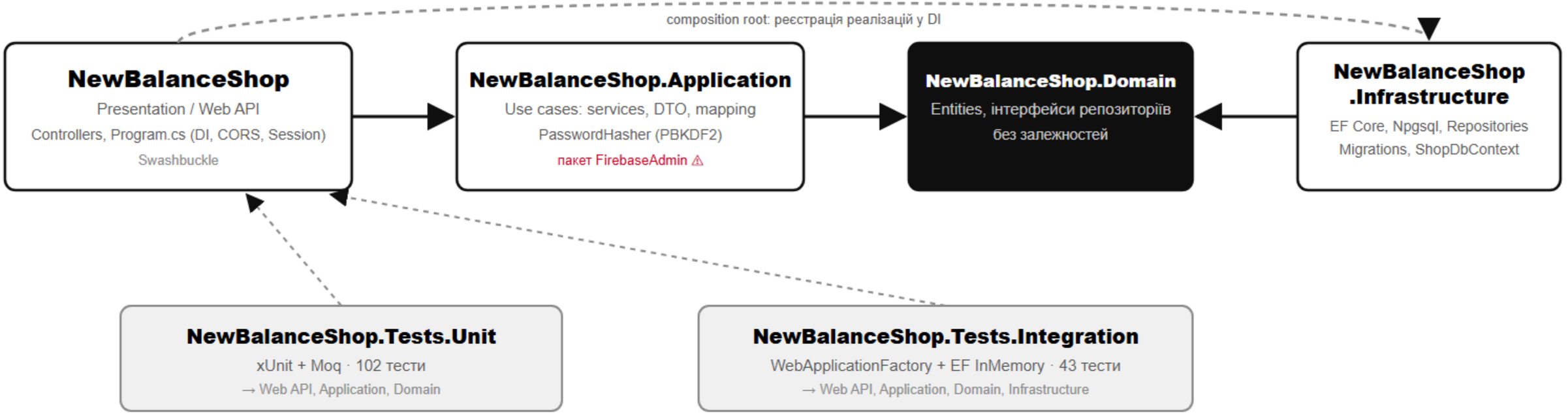


Адреси: фронтенд newbalanceshop-frontend.vercel.app, API newbalanceshop-api.onrender.com.

Чиста архітектура: проєкти та зв'язки

4 проєкти рішення NewBalanceShop.slnx + 2 тестові · стрілка = ProjectReference (залежність)

New Balance Shop



Правило залежностей: усі стрілки між шарами спрямовані до Domain

Що дотримано

- Domain не посилається ні на що (немає ProjectReference).
- Infrastructure залежить лише від Domain: контракти репозиторіїв лежать у Domain.
- Controllers викликають лише сервіси Application, EF Core у них немає.

Відоме відхилення і план виправлення

Application має пакет FirebaseAdmin: перевірка Google-токена лежить у AuthService. План: інтерфейс **IGoogleTokenVerifier** у Domain/Application, реалізація в Infrastructure.

Використані патерни

Кожен патерн розв'язує конкретну задачу проєкту

Repository

`IRepository<T> → Repository<T>`

Доступ до даних сховано за інтерфейсом у Domain.

```
public interface IProductRepository
  : IRepository<Product>
{ Task<List<Product>> GetByCategoryAsync(int id); }
```

Навіщо: сервіси не знають про EF Core; у юніт-тестах репозиторій підміняється Moq.

Dependency Injection

`Program.cs`

Усі залежності передаються через конструктор, реєстрація в одному місці.

```
builder.Services
 .AddScoped<IProductService, ProductService>();
```

Навіщо: слабка зв'язаність; в інтеграційних тестах підміняємо БД на InMemory.

Service Layer (Use Cases)

`Application/Services`

Бізнес-логіка в сервісах, контролер лише приймає й повертає HTTP.

```
OrderService.CreateAsync:
 перевірка складу, UnitPrice,
 TotalPrice, зменшення Stock
```

Навіщо: логіку можна тестувати без HTTP і без БД.

DTO + Mapper

`Application/DTO, Mapping`

API повертає DTO, а не сутності EF; мапінг у extension-методах.

```
public static CustomerDto ToDto(
  this Customer c) => new() { … };
```

Навіщо: PasswordHash ніколи не потрапляє у відповідь (є тест на це).

Middleware Pipeline

`Program.cs`

Запит проходить послідовність компонентів.

```
UseSwagger → UseCors
 → UseSession → MapControllers
```

Навіщо: CORS із cookie й сесія для кошика та логіну налаштовані централізовано.

Session State

`CartController, AuthController`

Кошик гостя й ID залогіненого покупця живуть у серверній сесії.

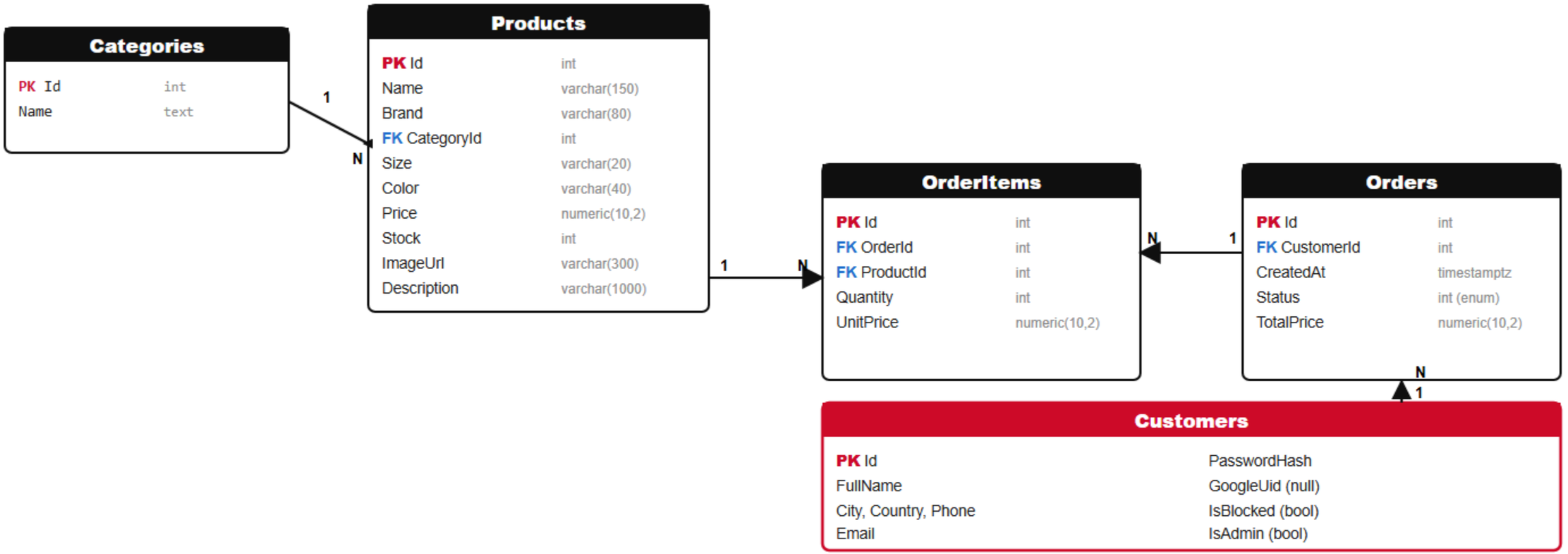
```
HttpContext.Session
 .SetInt32("customerId", id);
```

Навіщо: кошик не пишемо в БД до оформлення; логін без токенів.

База даних: ER-діаграма

PostgreSQL (Render) · 5 таблиць · EF Core Migrations + seed

New Balance Shop



Чому PostgreSQL

- Реляційні зв'язки замовлення → позиції → товар, потрібна транзакційність.
- Безкоштовний managed-інстанс на Render, є провайдер Npgsql для EF Core.

Структура

3НФ: ціна позиції (UnitPrice) фіксується на момент замовлення; Customer не дублює дані замовлень. Індекси на всіх зовнішніх ключах.

Що покращити

Унікальний індекс на Customers.Email; м'яке видалення замість каскадного (зараз видалення клієнта прибирає його замовлення).

Authentication / Authorization та безпека

Як зберігаємо паролі, як перевіряємо права і що ще треба закрити

New Balance Shop

Аутентифікація: 3 способи входу

1. Email + пароль: `/api/Auth/register, login`
2. Google: Firebase ID-токен, API перевіряє підпис
3. GitHub OAuth (authorization code flow)

→ Сесія: cookie HttpOnly, Secure, таймаут 30 хв

```
Пароль: PBKDF2-HMAC-SHA256
100 000 ітерацій · сіль 16 байт · хеш 32 байти
формат: ітерації.сіль.хеш · FixedTimeEquals
Google-користувачі: пароль не зберігається
```

Авторизація і захист

ЗАГРОЗА	ЗАХИСТ	
SQL Injection	EF Core, параметризовані запити	**OK**
XSS	React екранує вивід	**OK**
Паролі	Hash + salt, не SHA256(password)	**OK**
Адмін-функції	Перевірка IsAdmin: 401 / 403	**OK**
Чужий профіль	Тільки власник або адмін	**OK**
Секрети	DATABASE_URL, Firebase JSON у env	**OK**
BOLA (замовлення)	customerId у запиті довільний	**Ризик**
CRUD товарів	Без перевірки ролі	**Ризик**
CORS / CSRF	Будь-який origin з cookie	**Спрощено**

План закриття ризиків

- **BOLA:** брати customerId із сесії в /api/Orders і checkout, а не з параметра.
- **Товари:** POST/PUT/DELETE лише для IsAdmin (FR-04–06).
- **CORS:** білий список origin: Vercel-домен і localhost.
- **Сесії:** зараз у пам'яті процесу; перенести в Redis або БД.
- **Brute-force:** rate limiting на login.
- **Email:** унікальний індекс у БД.

Testing & Logging

xUnit · Moq · WebApplicationFactory · EF InMemory · coverlet

Піраміда тестів

145 / 145 passed

Integration · 43
повний HTTP-конвеєр, EF InMemory

Unit · 102
сервіси, мапери, контролери (Moq)

Unit (NewBalanceShop.Tests.Unit)
Product/Order/Auth/CustomerService · PasswordHasher (5) · Mapping · контролери: Products, Orders, Auth, Customers, GithubAuth

Integration (NewBalanceShop.Tests.Integration)
Products 10 · Cart 7 · Orders 6 · Auth 8 · Customers 12; кожен тест на власній InMemory-БД

Покриття: 86,7%
Infrastructure 95,8% · Application 90,2% · Domain 84,6% · Web API 82,0%

Що знайшли тести (реальні баги)

- **Forbid() давав 500** замість 403, бо схему автентифікації не реєстровано; тепер явні 401/403.
- **Secure-cookie ламала сесію** у тестах по http; у середовищі Testing вимкнено.
- **Два провайдери БД** при підміні Npgsql на InMemory; усунуто видаленням конфігурації контексту.

Logging

Використовується вбудований **ILogger** ASP.NET Core: запити, EF Core і помилки йдуть у консоль/логи Render. Окремих логів бізнес-подій немає.

План покращення

- Middleware глобальної обробки помилок + ILogger у сервісах.
- Запуск тестів у GitHub Actions на кожен push.

Ключові метрики

Фактичні цифри на 05.10.2026, виміряно з репозиторіїв і прогону тестів

New Balance Shop

12/13
ВИМОГ SRS ПОВНІСТЮ

29
REST-ЕНДПОІНТІВ

145
ТЕСТІВ (102 UNIT + 43 INTEGRATION)

86,7 %
CODE COVERAGE (РЯДКИ)

30
КОМІТІВ (БЕКЕНД 24 + ФРОНТЕНД 6)

3 тиж.
АКТИВНОЇ РОЗРОБКИ: 14.09 – 05.10

1,6k
РЯДКІВ C# У БЕКЕНДІ (БЕЗ ТЕСТІВ І МІГРАЦІЙ)

2,2k
РЯДКІВ C# У ТЕСТАХ

Покриття тестами за проєктами (unit + integration, злиті звіти coverlet)



804 рядки, 697 покрито. Без міграцій і тестових проєктів. Команда: dotnet test --collect:"XPlat Code Coverage".

Scrum: спринти, беклог, канбан

Jira Scrum board SCRUM · newbalance-team1.atlassian.net

New Balance Shop

Спринти

СПРИНТ	ДАТИ	ЦІЛЬ
1	до 14.09	SRS, архітектура, CRUD
2	31.08–06.09	Початок розробки, SCRUM-ролі
3	07.09–13.09	Замовлення, Customer
4	14.09–20.09	Деплой, фронтенд, i18n
5	21.09–27.09	MVP, авторизація, 4 проєкти
6	28.09–04.10	Кабінет, адмін, тести

Team Lead — Панчук Вадим; команда розробки — обидва учасники.

Канбан-дошка (стан на 05.10.2026)

BACKLOG 6

- CI: GitHub Actions (build + test)
- Кешування категорій (IMemoryCache)
- Логування й обробка помилок (middleware)
- DocFX для публічних методів
- Форми логіну/реєстрації й адмін-панель у React
- Фільтр за брендом, ціною, розміром (FR-03)

IN PROGRESS 2

- Презентація й сценарій доповіді
- Матеріали до захисту (скріни, схеми)

DONE 12

- SRS, UML, архітектура
- CRUD товарів
- Замовлення та кошик
- PostgreSQL + Docker + Render
- React-фронтенд (Vercel)
- Локалізація UA/EN/FR
- Clean Architecture: 4 проєкти
- Авторизація: сесії, GitHub, Google
- Кабінет покупця (FR-10, 11)
- Адмін-функції (FR-12, 13)
- 145 автотестів, покриття 86,7%
- Scalar-документація API

Стан зібрано за репозиторієм. Скрін самої Jira-дошки додається окремо (потрібен вхід в Atlassian).

Tools & Workflow

Інструменти розробки, деплою та командної роботи

БЕКЕНД І ДАНІ

- .NET 9 / ASP.NET Core
- PostgreSQL
- EF Core + Npgsql
- Swagger / Scalar
- Firebase Auth

ФРОНТЕНД

- React
- Vite
- i18next (UA/EN/FR)

ТЕСТУВАННЯ

- xUnit
- Moq
- WebAppFactory
- coverlet

ПРОЦЕС І ДЕПЛОЙ

- GitHub
- Jira (Scrum)
- Docker
- Render (API + БД)
- Vercel (фронтенд)

Код → **git push** → **GitHub** → **Render збирає Docker** / **Vercel збирає React** → **онлайн**

Портфоліо-цінність

Чому проєкт важливий для розробників, потенціал для аудиторії та монетизація

1

Для нас як розробників

- Повний цикл: вимоги, архітектура, БД, API, фронтенд, деплой.
- Clean Architecture із 4 проєктів і 145 автотестами.
- Авторизація трьома способами: пароль, Google, GitHub.
- Працює онлайн, є що показати роботодавцю.

2

Потенціал для ЦА

- Швидка покупка без обов'язкової реєстрації.
- Кабінет: профіль та історія замовлень.
- Інтерфейс трьома мовами: UA, EN, FR.
- Каталог легко розширити: нові категорії, бренди, фільтри.

3

Можливості монетизації

- Партнерська комісія з продажів бренду.
- Платна доставка й прискорене оформлення.
- Інтеграція оплати (картка, Apple/Google Pay).
- Рекламні й промо-місця для нових колекцій.
- Шаблон для малого бізнесу: готовий магазин «під ключ».

КЛЮЧОВА ДУМКА ПРОЄКТУ

Від каталогу до замовлення за кілька кліків, без зайвої реєстрації.

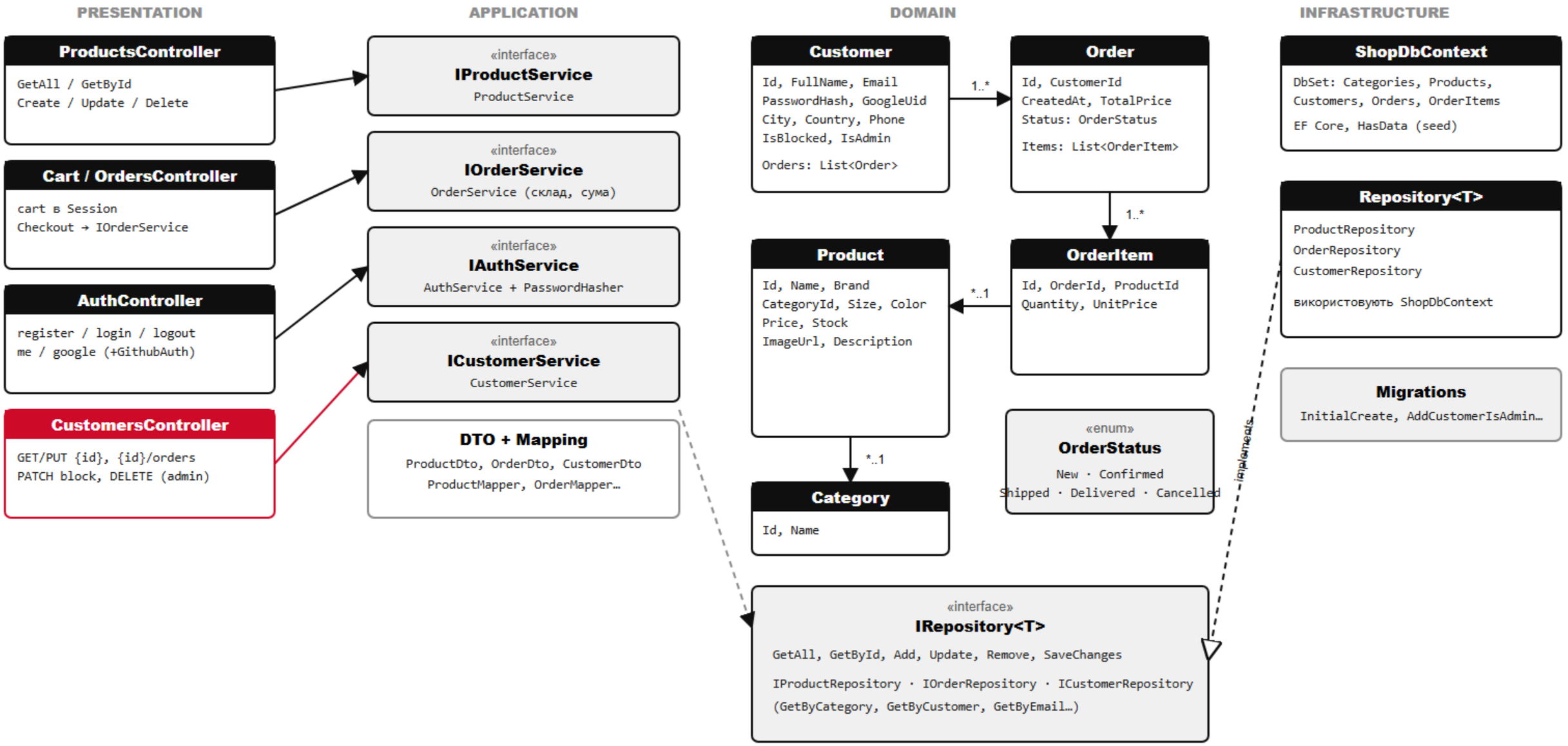
Ваші питання?

New Balance Shop · Team 1 · дякуємо за увагу

Діаграма класів

Ключові класи й інтерфейси за шарами (спрощено)

New Balance Shop

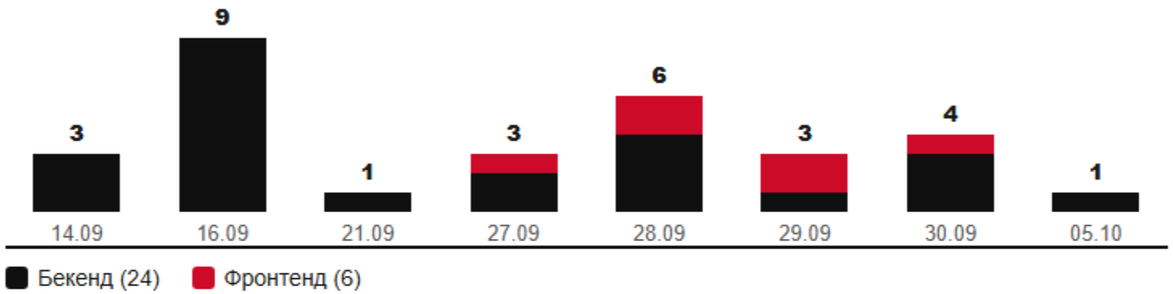


Командна робота в Git

Коміти, гілки та внесок учасників · репозиторії бекенду й фронтенду

New Balance Shop

Коміти за днями (усього 30)



Гілки репозиторію бекенду

ГІЛКА	ПРИЗНАЧЕННЯ	СТАН
`master`	Основна гілка, деплой на Render	23 коміти
`feature/pl-dal-layers`	ДЗ: PL + DAL без BLL (PLDAL-Task/)	1 коміт, окремо від прод
`feature/customers-cabinet`	Кабінет покупця	створена; роботу вели в master

Merge-комітів немає: робота велась у master (trunk-based), фіча-гілки не використовувались на повну.

Внесок учасників

УЧАСНИК	РОЛЬ	КОМІТИ	ЩО ЗРОБЛЕНО
Ноцколіні Рафаїл	Full-stack розробник	**30 (100%)**	Бекенд: архітектура, API, БД, авторизація, тести; фронтенд React; деплой; документація
Панчук Вадим	Team Lead	**0**	Запрошений у Jira як учасник; коміти в репозиторії відсутні

⚠ Внесок Team Lead у планування, Jira-дошку й рев'ю не відображено в git. Доповнити вручну перед захистом.

Документація

README, документація API (Scalar / OpenAPI), архітектурні й користувацькі документи

Що є в репозиторії

ДОКУМЕНТ	ПРИЗНАЧЕННЯ	СТАН
README.md	Опис, стек, запуск, ендпоінти	**Частково, оновити стек і ендпоінти**
SRS.md	Вимоги, use cases, NFR, матриця трасування	**є**
docs/ARCHITECTURE.md	Clean Architecture, UML класів	**є**
docs/STRUCTURE.md	Структура бекенду, фронтенду, БД	**є**
docs/api/index.html	Документація REST API на Scalar з OpenAPI	**є**
Swagger UI (/swagger)	Інтерактивний API-провайдер на Render	**є**
docs/user_guide.html	Інструкція для покупця	**є**
docs/test_report.html	Звіт про тестування (145 тестів)	**є**
DocFX на GitHub Pages	Документація публічних методів із коментарів коду	**Немає, у плані**

```
NewBalanceShop/
├─ README.md · SRS.md · TEAM.md · BACKLOG.md
├─ docs/
│  ├─ ARCHITECTURE.md · STRUCTURE.md
│  ├─ api/  index.html (Scalar) + openapi.json
│  ├─ architecture/  структура back/front/DB
│  ├─ db/  postgres-tables.png
│  ├─ user_guide.html
│  └─ test_report.html
├─ defense-materials/  слайди до захисту
└─ Dockerfile · render.yaml
```

Документація API

Специфікація OpenAPI генерується Swashbuckle, HTML-сторінка побудована на Scalar. Ендпоінти згруповані: Products, Cart, Orders, Auth, Customers.